

SAMPLING AND FLOW/SCORE MATCHING

CHENYANG SUN

ABSTRACT. This is intended as a simple and concise introduction to sampling, flow matching, and score matching in the context of generative models, based on [these lecture notes](#). The content of this note corresponds roughly to the first 4 chapters. Direct questions, comments, suggestions, or corrections to cs4472@columbia.edu.

A premise of generative modeling is that a collection of objects, such as dog pictures, forms a distribution on some background space, in this case the space of all images that can be displayed on your screen. The objective of generative modeling is bipartite: to “learn”, i.e., infer, the underlying distribution from samples drawn from it, and to generate a sample from that distribution. Conveniently, both problems can be formulated and treated in a single framework based on ODEs and SDEs.

CONTENTS

1. Sampling	1
1.1. Flow to δ_z	2
1.2. Flow to p^* via ODE	2
1.3. Flow to p^* via SDE	3
2. Inference via Flow and Score Matching	4
2.1. Loss function for flow and score matching	4
2.2. Score-to-flow conversion	5

1. SAMPLING

We first tackle the problem of sampling: suppose that we know explicitly a distribution p^* on $\mathbb{R}^d$; how do we generate a sample from it? We consider an ODE

$$dX_t = u_t(X_t)dt, \quad t \in [0, 1], \quad X_t \in \mathbb{R}^d, \quad u_t : \mathbb{R}^d \rightarrow \mathbb{R}^d, \quad (1.1)$$

where we initialize X_0 with some easy-to-sample distribution, say $p_0 = N(0, 1)$, and let that probability distribution flow according to the time-varying vector field u_t . The goal is to find u_t such that X_1 has the target distribution p^* . This way, we could sample $X_0 \sim N(0, 1)$, feed the sample into the ODE, and arrive at a sample of $X_1 \sim p^*$.

1.1. Flow to δ_z . We first solve the problem in the simplest case, where $p^* = \delta_z$ for some $z \in \mathbb{R}^d$. This seems completely unnecessary, as we know how to sample from δ_z , but the general case is only a slight extension via averaging z according to p^* .

Denote by p_t the distribution of X_t . One could think of $\{p_t\}_{t \in [0,1]}$ as providing an interpolation between p_0 and p_1 . We could choose the simplest interpolation that comes to mind, one that moves both mean and deviation at a linear rate: $p_t = N(tz, (1-t)^2)$, which is attained by setting

$$X_t = tz + (1-t)X_0. \quad (1.2)$$

To find u_t , we reverse-engineer the equation (1.1) from its solution (1.2). Taking the derivative of the solution gives $dX_t = (z - X_0)dt$, and substituting away X_0 according to (1.2) yields

$$dX_t = \left(z - \frac{X_t - tz}{1-t} \right) dt = \left(\frac{z - X_t}{1-t} \right) dt, \quad (1.3)$$

$$u_t(x) = \frac{z - x}{1-t}. \quad (1.4)$$

1.2. Flow to p^* via ODE. For general distributions p^* , we rename p_t from above to include the hidden parameter z , as $p_t(\cdot|z)$, and think of it as the conditional distribution of p_t (associated with the new p^*), on $X_1 = z$. Similarly, we rename u_t from before as $u_t(\cdot|z)$. Averaging the conditional density $p(\cdot|z)$ according to $z \sim p^*$ yields the marginal distribution

$$p_t(\cdot) := \int_{\mathbb{R}^d} p_t(\cdot|z) dp^*(z), \quad (1.5)$$

which is easily checked to have p_0 and p^* as endpoints in time. We now have the probability path p_t , and need to compute the drift u_t that produces it.

Recall that the *Fokker-Planck equation*

$$\partial_t p_t(x) = -\operatorname{div}(p_t(x)u_t(x)) + \frac{\sigma_t^2}{2} \operatorname{div}(\nabla p_t(x)) \quad (1.6)$$

relates the drift u_t to the distribution $X_t \sim p_t$ in the SDE $dX_t = u_t(X_t)dt + \sigma_t dW_t$. Note that ∂_t denotes partial derivative with respect to t , and that gradient and divergence operate on the collection of spatial variables x . Setting $\sigma_t \equiv 0$ yields the *continuity equation*:

$$\partial_t p_t(x) = -\operatorname{div}(p_t(x)u_t(x)) \quad (1.7)$$

for our ODE. Plugging the decomposition (1.5) into the continuity equation yields

$$\partial_t p_t(x) = \partial_t \int_{\mathbb{R}^d} p_t(x|z) dp^*(z) \quad (1.8)$$

$$= \int_{\mathbb{R}^d} \partial_t p_t(x|z) dp^*(z) \quad (1.9)$$

$$= \int_{\mathbb{R}^d} -\operatorname{div}(p_t(x|z)u_t(x|z)) dp^*(z) \quad (1.10)$$

$$= -\operatorname{div} \int_{\mathbb{R}^d} p_t(x|z)u_t(x|z) dp^*(z) \quad (1.11)$$

$$= -\operatorname{div} \left(p_t(x) \frac{1}{p_t(x)} \int_{\mathbb{R}^d} p_t(x|z)u_t(x|z) dp^*(z) \right), \quad (1.12)$$

thus

$$u_t(x) = \frac{1}{p_t(x)} \int_{\mathbb{R}^d} p_t(x|z)u_t(x|z) dp^*(z). \quad (1.13)$$

Question (PJ, Leo). *The ODE induces a transport plan between the distributions p_0 and p^* on $\mathbb{R}^d$. Does there exist a natural cost function on $\mathbb{R}^d$ such that this plan is optimal in the sense of Monge? In the sense of Kantorovich?*

1.3. Flow to p^* via SDE. In the case of an SDE $dX_t = v_t(X_t)dt + \sigma_t dW_t$, we use the full Fokker-Planck equation to find the drift v_t such that X_t has distribution p_t . (Note that we call the SDE drift v_t to avoid conflation with the ODE drift u_t we computed.) We set

$$\partial_t p_t(x) = -\operatorname{div}(p_t(x)u_t(x)) = -\operatorname{div}(p_t(x)v_t(x)) + \frac{\sigma_t^2}{2} \operatorname{div}(\nabla p_t(x)), \quad (1.14)$$

and obtain

$$-\operatorname{div}(p_t(x)v_t(x)) = -\operatorname{div}(p_t(x)u_t(x)) - \frac{\sigma_t^2}{2} \operatorname{div}(\nabla p_t(x)) \quad (1.15)$$

$$= -\operatorname{div} \left(p_t(x) \left(u_t(x) - \frac{\sigma_t^2}{2} \frac{\nabla p_t(x)}{p_t(x)} \right) \right), \quad (1.16)$$

that is,

$$v_t(x) = u_t(x) - \frac{\sigma_t^2}{2} \frac{\nabla p_t(x)}{p_t(x)} \quad (1.17)$$

$$= u_t(x) - \frac{\sigma_t^2}{2} \nabla \log p_t(x). \quad (1.18)$$

The term $\nabla \log p_t(x)$ is known in the literature as the *score function*. Recall the decomposition (1.13) of the drift $u_t(x)$ in terms of conditional flow $u_t(x|z)$; the same decomposition

applies to the score: we have

$$\nabla \log p_t(x) = \frac{\nabla p_t(x)}{p_t(x)} \quad (1.19)$$

$$= \frac{1}{p_t(x)} \nabla \int_{\mathbb{R}^d} p_t(x|z) dp^*(z) \quad (1.20)$$

$$= \frac{1}{p_t(x)} \int_{\mathbb{R}^d} \nabla p_t(x|z) dp^*(z) \quad (1.21)$$

$$= \frac{1}{p_t(x)} \int_{\mathbb{R}^d} \frac{\nabla p_t(x|z)}{p_t(x|z)} p_t(x|z) dp^*(z) \quad (1.22)$$

$$= \frac{1}{p_t(x)} \int_{\mathbb{R}^d} \nabla \log p_t(x|z) p_t(x|z) dp^*(z). \quad (1.23)$$

The identity of these two decompositions will become important in score matching.

Question. *Evan pointed out that historically, SDE-based score matching—which seems to be more computationally intensive—came before ODE-based flow matching. This seems unusual and worth a further look. Additionally, try to find examples where the additional complexity of SDEs is justified by better performance.*

2. INFERENCE VIA FLOW AND SCORE MATCHING

Previously, we have resolved the problem of generating a sample from a given objective distribution p^* : we draw a sample from $N(0, 1)$, and pass it through a carefully-engineered ODE/SDE, and retrieve the result, which now has the desired distribution.

However, we don't know what the objective distribution p^* is—all we have is a collection of samples drawn from it, and need to estimate what p^* is given this incomplete information. Instead of estimating p^* directly, we estimate the drift u_t , known as *flow matching*. This is equivalent to estimating p^* , since p^* and u_t can be obtained from each other, and we bypass the process of computing u_t from p^* described in the previous section.

2.1. Loss function for flow and score matching. In the literature, an approximant to u_t is often written as u_t^θ , where θ is a set of parameters describing u_t^θ in the context of neural networks. Here, we outsource the problem of training the network, i.e., finding the optimal parameters, and merely formulate the pertinent optimization problems. Define the *flow matching loss* function,

$$F(\theta) := \mathbb{E} (\|u_T^\theta(X) - u_T(X)\|^2), \quad (2.1)$$

where $T \sim \text{Unif}[0, 1]$ and $X \sim p_T$; in other words, the mean squared-error between the estimator and true flow. Alternatively, we can sample $X \sim p_T$ by first sampling $Z \sim p^*$, and then sampling the conditional distribution $X \sim p_T(\cdot|Z)$. An advantage of this representation is that the training dataset provides i.i.d. samples of $Z \sim p^*$, while sampling a Gaussian conditional distribution is easy.

Unfortunately, the quantity $u_T(X)$ is intractable, as the decomposition (1.13) requires knowledge of p_t , which itself requires knowledge of p^* . One might naively replace $u_T(X)$ by $u_T(X|Z)$ to obtain the *conditional flow matching loss*

$$F_c(\theta) := \mathbb{E} (\|u_T^\theta(X) - u_T(X|Z)\|^2), \quad (2.2)$$

which becomes more tractable, but it is unclear how F_c relates to the original loss function F . Amazingly, the minimizations of F and F_c are equivalent, in the sense that F and F_c differ by a constant. Expanding the square yields

$$F(\theta) - F_c(\theta) = \mathbb{E} (\|u_T(X)\|^2 - \|u_T(X|Z)\|^2) \quad (2.3)$$

$$+ 2 (\mathbb{E} (u_T^\theta(X)^\top u_T(X|Z)) - \mathbb{E} (u_T^\theta(X)^\top u_T(X))) , \quad (2.4)$$

where the first expectation is constant in θ , and the rest vanishes:

$$\mathbb{E} (u_T^\theta(X)^\top u_T(X|Z)) = \int_{t \in [0,1]} \int_{x \in \mathbb{R}^d} \int_{z \in \mathbb{R}^d} u_t^\theta(x)^\top u_t(x|z) dp_{Z,X,T}(z, x, t) \quad (2.5)$$

$$= \int_{t \in [0,1]} \int_{x \in \mathbb{R}^d} \int_{z \in \mathbb{R}^d} u_t^\theta(x)^\top u_t(x|z) p_t(x|z) dp^*(z) dx dt \quad (2.6)$$

$$= \int_{t \in [0,1]} \int_{x \in \mathbb{R}^d} u_t^\theta(x)^\top p_t(x) \frac{1}{p_t(x)} \int_{z \in \mathbb{R}^d} u_t(x|z) p_t(x|z) dp^*(z) dx dt \quad (2.7)$$

$$= \int_{t \in [0,1]} \int_{x \in \mathbb{R}^d} u_t^\theta(x)^\top u_t(x) p_t(x) dx dt \quad (2.8)$$

$$= \mathbb{E} (u_T^\theta(X)^\top u_T(X)) . \quad (2.9)$$

For SDEs, we need to perform *score matching* to obtain the drift v_t in (1.17). For some approximant $s_t^\theta(x)$ to the true score $\nabla \log p_t(x)$, we similarly define the marginal and conditional *score matching loss* functions:

$$S(\theta) := \mathbb{E} (\|s_T^\theta(X) - \nabla \log p_T(X)\|^2), \quad (2.10)$$

$$S_c(\theta) := \mathbb{E} (\|s_T^\theta(X) - \nabla \log p_T(X|Z)\|^2). \quad (2.11)$$

Recall that the equivalence of F and F_c relied only on the decomposition (1.13) of (marginal) flow u_t in terms of conditional flow $u_t(\cdot|z)$. Since the same decomposition holds for (marginal) score in terms of conditional score, S and S_c are equivalent objective functions by the same argument.

2.2. Score-to-flow conversion. The form of the SDE drift (1.17), at first glance, suggests that we need to estimate the ODE drift u_t in addition to the score $\nabla \log p_t(x)$, in order to approximate the SDE drift v_t . Fortunately, u_t can be calculated from the score. We first handle the conditional version: since $p_t(\cdot|z) \sim N(tz, (1-t)^2)$, we have

$$\nabla \log p_t(x|z) = \frac{tz - x}{(1-t)^2}, \quad (2.12)$$

and recalling (1.3), we have

$$u_t(x|z) = \frac{z - x}{1 - t} \quad (2.13)$$

$$= \frac{tz - tx}{t(1 - t)} \quad (2.14)$$

$$= \frac{tz - x}{t(1 - t)} + \frac{x - tx}{t(1 - t)} \quad (2.15)$$

$$= \frac{1 - t}{t} \nabla \log p_t(x|z) + \frac{x}{t}. \quad (2.16)$$

The marginal flow can be obtained from the marginal score the exact same way, taking advantage yet again of the fact that the flow and score decompositions (1.13), (1.19) are the same:

$$u_t(x) = \frac{1}{p_t(x)} \int_{\mathbb{R}^d} p_t(x|z) u_t(x|z) dp^*(z) \quad (2.17)$$

$$= \frac{1}{p_t(x)} \int_{\mathbb{R}^d} p_t(x|z) \left(\frac{1 - t}{t} \nabla \log p_t(x|z) + \frac{x}{t} \right) dp^*(z) \quad (2.18)$$

$$= \frac{1 - t}{t} \frac{1}{p_t(x)} \int_{\mathbb{R}^d} p_t(x|z) \nabla \log p_t(x|z) dp^*(z) + \frac{x}{t} \frac{1}{p_t(x)} \int_{\mathbb{R}^d} p_t(x|z) dp^*(z) \quad (2.19)$$

$$= \frac{1 - t}{t} \nabla \log p_t(x) + \frac{x}{t}. \quad (2.20)$$